Previously, a pure-Erlang web server called Cowboy[6] was the default web server.

Integrate with Ecto to Provide Data Access

Ecto works well with Phoenix. The Phoenix/Ecto[7] package integrates Ecto changesets and structures into Phoenix's form helpers. This creates a seamless experience when using Ecto changesets in Phoenix controllers and views.

Provide Real-Time Foundations

Phoenix provides the foundation for creating real-time web applications via Channels. Phoenix provides WebSocket support as well as helpers that make it easy to write real-time applications. The Phoenix LiveView[8] package takes this a step further with server-rendered real-time HTML.

You may have noticed something: these features are all web-request related. There's no mailer, job system, database, or storage code found in Phoenix. This highlights a fundamental difference between Rails and Phoenix. Let's dig into that.

External Web Servers



BEAM-based web servers are scalable enough to use without a proxy in front of them. It's a common pattern in many other languages (including Ruby) to use a proxy (nginx, Apache, and so on) that can handle a large amount of concurrent connections. This is due to concurrency limitations in Ruby and other languages.

Due to the BEAM's concurrency model, it's possible to run a production-grade web application without a proxy in front of it. This simplifies deployments without sacrificing safety.

How Does Phoenix Compare to Rails?

It has been a goal from the beginning of Rails that "Rails is omakase."[9] Omakase means "I leave the details up to you," so Rails intends to provide all of the features needed to create a high-quality application. Ruby has a rich gem ecosystem, but Rails has absorbed and recreated popular gems into the framework itself. The all-in-one nature of Rails—and how the framework

6. https://github.com/ninenines/cowboy
7. https://hexdocs.pm/phoenix_ecto/main.html
8. https://github.com/phoenixframework/phoenix_live_view
9. https://dhh.dk/2012/rails-is-omakase.html